Module

1

Algorithm Analysis and Graph Foundations

This module introduces core algorithm analysis techniques and advanced graph concepts, covering complexity analysis, recurrence solving, graph traversal, topological ordering, strongly connected components, and bridge detection through practical problem-solving assignments.

1.1

Course Orientation | Design and Analysis of Algorithms

Introduce the Design and Analysis of Algorithms course structure, five algorithmic paradigms, and the role of complexity analysis in engineering.

1.2

Optional

Algorithm Paradigm Classifier

This lesson provides an easier way to classify a given algorithm into one of the five paradigms and justify the classification.

1.3

Asymptotic Analysis and Amortized Complexity

Formalize Big-Θ, Big-Ω, Big-O notation and compute the amortized cost of dynamic array push using the aggregate method.

1.4

Optional

Assignment | Tight Bound Proof Challenge

Prove that a given function satisfies a specific Big-Θ bound by finding witness constants c1, c2, and n0.

1.5

Recurrence Relations and Master Theorem

Set up recurrence relations for recursive algorithms and solve them using substitution, recursion tree, and the Master Theorem.

1.6

Optional

Assignment | Recurrence Derivation: Pow(x, n)

Write the recurrence for fast exponentiation and solve it using the Master Theorem.

1.7

Revisiting Graph Foundations

Review graph representations and formalise BFS and DFS with O(V+E) complexity proofs and DFS edge classification.

1.8

Optional

Assignment | Bipartite Graph Check

Extend BFS and DFS traversal techniques to solve the graph 2-coloring problem, and understand the fundamental relationship between bipartite graphs and the absence of odd-length cy

1.9

Optional

Assignment | Unique Paths

Solve the unique paths in a graph without any memorization.

1.10

Assignment | Rotting Oranges

Map 2D grid matrix problems to graph traversal paradigms. Implement multi-source BFS for simultaneous level-by-step propagation and standard DFS for identifying connected component

1.11

Assignment | Topological Sorting

Master the concept of linear ordering in Directed Acyclic Graphs (DAGs) and understand the internal mechanics of Kahn's Algorithm (BFS-based) and finish-time tracking (DFS-based) f

1.12

Assignment | Course Schedule

Implement SCC or topological DFS to detect cycles and determine valid orderings in a directed prerequisite graph.

1.13

Optional

Assignment | DFS Cycle Detection in Undirected Graph

Detect a cycle in an undirected graph using DFS by tracking the parent node to avoid false positives.

1.14

Kosaraju's Strongly Connected Components Algorithm

Implement Kosaraju's two-pass DFS algorithm to identify strongly connected components in a directed graph.

1.15

Tarjan's Algorithm and Bridge Detection

Implement Tarjan's single-pass DFS using low-link values to detect SCCs in directed graphs and bridges in undirected graphs.

1.16

Optional

Assignment | Articulation Points Detection

Apply the DFS low-link technique to find articulation points (cut vertices) in an undirected graph.

1.17

Assignment | Critical Connections in a network

Apply Tarjan's low-link DFS to identify bridge edges (critical connections) in an undirected network graph.

Module

2

Divide & Conquer

This module builds advanced algorithmic thinking through divide and conquer strategies, helping you design efficient solutions for complex searching, sorting, geometric, and optimization problems with strong analytical problem-solving skills.

2.1

Introduction to Divide and Conquer

Identify the three structural criteria of a D&C problem and set up recurrences using the T(n) = aT(n/b) + f(n) template.

2.2

Optional

Divide and Conquer vs Dynamic Programming Paradigm Boundary

Distinguish between D&C (independent subproblems) and DP (overlapping subproblems) for a set of classification problems.

2.3

Binary Search on Answer Space

Prove Binary Search correctness using a loop invariant and derive its O(log n) bound via T(n) = T(n/2) + O(1).

2.4

Optional

Assignment | Koko Eating Bananas

Apply Binary Search on a value range (answer space) rather than a sorted array to solve an optimization problem.

2.5

Assignment | Binary Search Variants

Apply Binary Search to boundary-finding and partition-based problems independently on Median of 2 Arrays, Peak Element.

2.6

Optional

Assignment | Median of Two Sorted Arrays

Implement the O(log(min(m,n))) Binary Search partition approach for the Median of Two Sorted Arrays problem.

2.7

Optional

Assignment | Find Minimum in Rotated Sorted Array

Apply Binary Search on a rotated sorted array to find the minimum element in O(log n).

2.8

Merge Sort & Recurrence Analysis

Derive Merge Sort's recurrence, prove the merge step is O(n), and augment the merge to count inversions without an extra pass.

2.9

Optional

Assignment | Sort List

Apply Merge Sort to a linked list where random access is unavailable, requiring a split-by-pointer approach.

2.10

Assignment | Count of Inversions

Implement the Merge Sort augmentation for counting inversions directly as a standalone coding problem.

2.11

Quick Sort Design and Analysis

Implement Lomuto partition, derive Quick Sort's best and worst case recurrences, and explain strategies that eliminate $O(n^2)$ behaviour.

2.12

Optional

Assignment | Quick Sort with 3-Way Partition

Implement the Dutch National Flag 3-way partition to handle arrays with many duplicate elements efficiently.

2.13

Assignment | Quick Sort Optimal Approach

Implement Quick Sort with random pivot and 3-way partition to avoid worst-case $O(n^2)$ and pass all test cases.

2.14

Optional

Assignment | Wiggle Sort

Rearrange an array such that A[0] ≤ A[1] ≥ A[2] ≤ A[3] … using Quick Select for O(n) median finding.

2.15

Optional

Assignment | K Closest Points to Origin

Use Quick Select to find the k closest points to the origin in O(n) expected time without full sorting.

2.16

Assignment | Closest Pair of Points

understand and Implement the Closest Pair of Points algorithm and the D&C approach to Maximum Subarray with written recurrence derivations.

2.17

Optional

Assignment | Fibonacci via Matrix Exponentiation

Compute the nth Fibonacci number in O(log n) using matrix exponentiation as a D&C application of fast power.

2.18

Assignment | Implementing Strassen's Matrix Multiplication

Implement Strassen's algorithm for n×n matrices and verify correctness against standard multiplication.

2.19

Optional

Assignment | Strassen Timing Comparison

Empirically compare the runtime of Strassen's algorithm vs standard matrix multiplication at different matrix sizes to find the crossover point.

2.20

The Skyline Problem

Design the Skyline Problem as a D&C merge problem that reconstructs a 2D contour in O(n log n) using two-pointer merging of recursive skylines.

2.21

Assignment | The Skyline Problem

Implement the Skyline Problem and Count of Smaller Numbers After Self using D&C merge approaches.

Module

3

Greedy Algorithms & Graph Optimization

This module explores greedy decision-making and graph optimization techniques, teaching how locally optimal choices lead to efficient global solutions in scheduling, compression, shortest-path, and minimum spanning tree problems.

3.1

Introduction to Greedy Algorithms

State the two formal conditions for greedy correctness, apply the exchange argument template, implement the Fractional Knapsack algorithm, and identify problems where greedy fails.

3.2

Optional

Evaluating Greedy Strategies

Evaluate proposed greedy strategies by examining their local choice, feasibility conditions, counterexamples, and correctness evidence before deciding whether to trust them.

3.3

Assignment | Activity Selection

Apply the Greedy Choice Property to interval scheduling, prove the earliest-finish-time criterion via an exchange argument, and implement the algorithm in O(n log n).

3.4

Optional

Assignment | Minimum meeting Rooms

Explore how changing Activity Selection from choosing meetings to scheduling every meeting requires a new greedy strategy. Use sorting and a min-priority queue to track active meetings and determine minimum rooms.

3.5

Assignment | Fractional Knapsack and Job Sequencing

Implement Fractional Knapsack independently and learn Job Sequencing with Deadlines before coding it, formally verifying the greedy choice property for each.

3.6

Optional

Fractional and 0/1 Knapsack Strategies

Compare Fractional and 0/1 Knapsack to identify how item divisibility changes valid choices and algorithm design. Examine why density-based greedy is optimal for fractional items but not guaranteed for whole items

3.7

Minimum Spanning Trees | Cut and Cycle Properties

State and prove the Cut Property and Cycle Property as the formal correctness foundations for all MST algorithms.

3.8

Disjoint Set Union

Implement Union-Find with path compression and union by rank, derive the amortised O(α(V)) complexity per operation, and identify its role as a cycle detection engine.

3.9

Optional

Assignment | Redundant Graph Edge Detection

Apply Disjoint Set Union to maintain connected components while processing graph edges in input order. Detect edge that forms cycle using representative equality and efficient union operations.

3.10

Kruskal's Algorithm

Implement Kruskal's algorithm using Union-Find from LU 3.5 and verify its correctness via the Cycle Property from LU 3.4.

3.11

Prim's Algorithm

Implement Prim's algorithm using a min-heap priority queue and verify its correctness via the Cut Property from MST.

3.12

Optional

Minimum Spanning Trees with Prim and Kruskal

Compare Prim and Kruskal by their greedy choices, data structures, graph representations, and disconnected behavior. Develop reasoning for selecting suitable MST approach without treating either algorithm as universally superior.

3.13

Dijkstra's Algorithm

Implement Dijkstra's algorithm as greedy shortest-path relaxation, prove why it is correct on non-negative graphs, and construct a counter-example demonstrating its failures.

3.14

Assignment | Dijkstra's Algorithm

Implement Dijkstra's algorithm on a real weighted graph problem and produce a written analysis of a negative-edge failure case.

3.15

Optional

Shortest Path Algorithms with Dijkstra

Evaluate graph conditions and Dijkstra implementations to determine whether standard Dijkstra is valid, invalid, or incorrectly implemented.

3.16

Huffman Coding

Design a Huffman encoding tree using a greedy min-heap strategy and prove that the resulting prefix-free code minimises total encoding length.

3.17

Assignment | Minimum Cost of Ropes

Implement Huffman Coding as a greedy min-heap algorithm to solve the Minimum Cost of Ropes problem and a second Huffman-variant problem independently.

3.18

Optional

Assignment | Job Sequencing with Deadlines

Apply greedy scheduling and DSU to maximize profit by assigning unit-time jobs to valid deadline slots efficiently.